

Build an E-commerce Customer Support AI Agent Chatbot

Objective

Build an **AI-powered customer support chatbot for an e-commerce platform** that can:

1. Answer **FAQ questions** using RAG.
2. Provide **order tracking** information from a database.
3. **Raise support tickets** when the customer has an issue.

The chatbot should use **LLM + tools** and expose API endpoints for interaction.

Technology Stack

You must use the following technologies:

- **Python**
 - **LangChain**
 - **Qdrant** (Vector Database)
 - **FastAPI or Flask** (for API)
 - **LangGraph** (optional)
 - **LLM Provider** (choose any one):
 - OpenAI
 - Gemini
 - Groq
-

Features to Implement

1. FAQ Question Answering (RAG)

Use **Retrieval Augmented Generation (RAG)** to answer frequently asked questions.

Requirements:

- Store FAQ documents in **Qdrant vector database**.
 - Implement **RAG as a tool**.
 - The workflow should be:
 1. LLM receives the question.
 2. LLM generates a **query for the RAG tool**.
 3. RAG retrieves relevant chunks from Qdrant.
 4. LLM generates the final answer using those chunks.
-

2. Order Tracking Tool

Allow users to check their **order status**.

Requirements:

- Ask the user for:
 - **Order ID**
 - **Mobile Number**
- Create a **dummy SQL database** containing order details.
- Query the database to fetch order status.
- Return order details to the user.

Example response:

```
Your order #12345 is currently shipped and will be delivered in 2 days.
```

3. Support Ticket System

If the user has a problem that cannot be resolved through FAQ or order tracking, the chatbot should suggest creating a **support ticket**.

Collect the following information from the user:

- Name
- Mobile Number
- Issue Description
- Order ID (if related to an order)

Process:

1. Collect all required details.
2. Create a **support ticket entry in the database**.
3. Generate a **Ticket ID**.
4. Send confirmation to the user.

Example confirmation:

```
Your support ticket has been created successfully.  
Ticket ID: TCKT1023  
Our support team will contact you shortly.
```

Chat History Management

- Maintain **chat history using session IDs**.
- Each user conversation must be tracked using a **session_id**.

API Requirements

You must implement the following endpoints using **FastAPI or Flask**.

1. `/chat` Endpoint

Used to interact with the chatbot.

Request Body

```
{
  "message": "Where is my order?",
  "session_id": "abc123"
}
```

Behavior

- Process the user message.
 - Use tools if required.
 - Maintain conversation history using `session_id`.
-

2. `/clear` Endpoint

Used to **clear chat history** for a session.

Request Body

```
{
  "session_id": "abc123"
}
```

Behavior

- Remove stored chat history for the given session.
-

Important Instructions

1. **Do NOT use prebuilt agents** such as:

```
create_agent()
```

1. You must build the agent **manually using tool binding**, such as:

- `bind_tools`
- custom tool calling logic

1. RAG must be implemented **as a tool**.
-

Expected System Flow

1. User asks a question.
 2. LLM decides:
 - Use **FAQ RAG**
 - Use **Order Tracking Tool**
 - Suggest **Support Ticket**
 3. Tools return results.
 4. LLM generates the final response.
-

Deliverables

Candidates should submit:

1. Complete **Python code**
 2. **Vector database setup (Qdrant)**
 3. **SQL database schema**
 4. **API implementation**
 5. **Instructions to run the project**
-